FORCED CONTEXT SWITCHING AND DETERMINISTIC LATENCY

Assignment 04

CSE323 Advanced Embedded Systems

Table of Contents

1.0 Executive Objective 2
2.0 System Architecture & Execution Flow 2
2.1 Task & ISR Hierarchy 2
3.0 Screenshot Verification & Comparative Analysis 2
4.0 The Role of the PendSV Exception 6
5.0 Interpretation & Engineering Trade-offs 6
6.0 Conclusion 7

Figure 1 Baseline Pre-Trigger State 3
Figure 2 Interrupt Timestamp Registration 3
Figure 3 Near-Zero Latency Validation 4
Figure 4 Interrupt Timestamp Registration (No Yield) 5
Figure 5 SysTick Boundary Latency 6

1.0 Executive Objective

The primary objective of this report is to empirically demonstrate the effect of forced context switching from within an Interrupt Service Routine (ISR) in a FreeRTOS environment. In standard operation, an RTOS scheduler evaluates task states at fixed intervals driven by the SysTick timer. If an ISR unblocks a high-priority task, that task typically must wait for the next SysTick boundary to execute. This assignment proves that by invoking the portYIELD_FROM_ISR() macro, an engineer can force an immediate, preemptive context switch upon ISR exit, reducing task awakening latency to near-zero.

2.0 System Architecture & Execution Flow

To cleanly measure the latency difference between the two scheduling paradigms (With Yield vs. Without Yield), a comparative timing system was architected.

2.1 Task & ISR Hierarchy

1. **vTriggerTask() (The Stimulus):** A background task responsible for generating software-triggered interrupts to predictably fire the ISR.
2. **vLowPriorityTask() (The Baseline):** An infinite loop task designed to increment gLowTaskIterations. Its continuous execution proves that the CPU is not artificially halted and the scheduler remains active during idle periods.
3. **GPIOF_Handler() (The Interceptor):** The target ISR. Upon execution, it:
    - Records the current RTOS tick into gLastIsrTick.
    - Unblocks the high-priority task using a synchronization primitive.
    - *Conditionally* invokes portYIELD_FROM_ISR() depending on the test phase.
4. **vHighPriorityTask() (The Response):** The task awakened by the ISR. Upon waking, it:
    - Records the current RTOS tick into gLastHighTaskTick.
    - Calculates the delay: gObservedLatencyTicks = gLastHighTaskTick - gLastIsrTick.
    - Increments gHighTaskRuns.

3.0 Screenshot Verification & Comparative Analysis

The Keil IDE's debugging environment and watch variables were utilized to capture the exact system state. The experiment was divided into two distinct runs to compare the RTOS behavior with and without the forced yield macro.

Phase A: Execution WITH portYIELD_FROM_ISR()

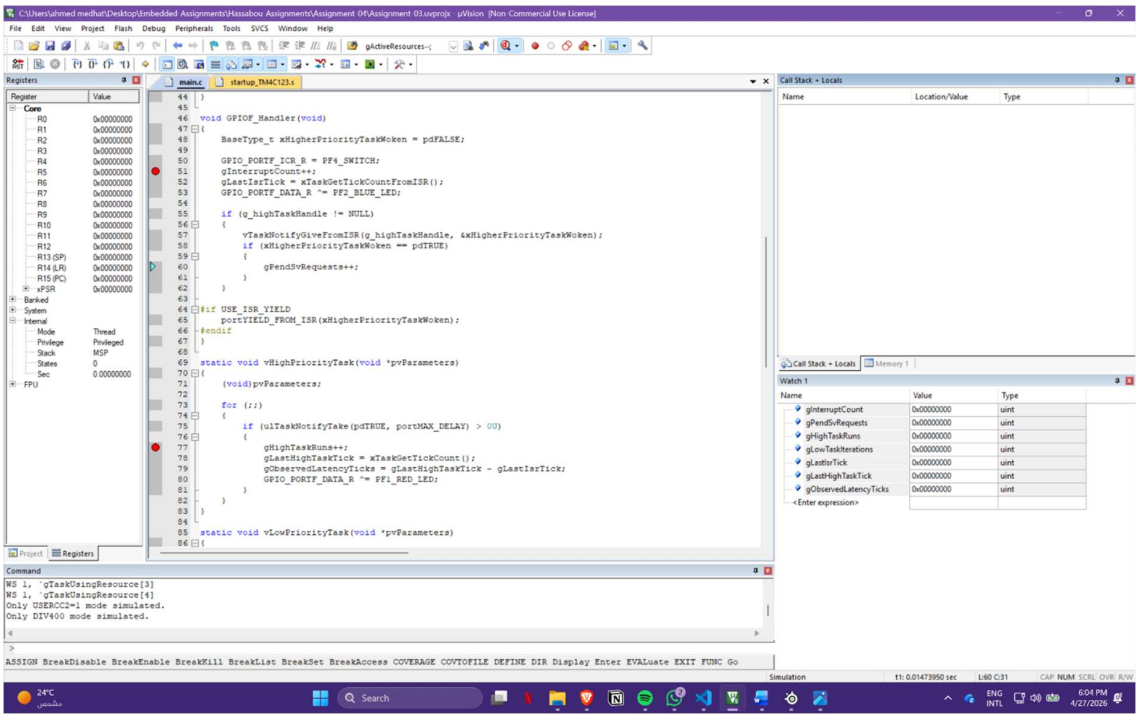

Figure 1 Baseline Pre-Trigger State

This capture shows the system running normally before the interrupt is triggered. gLowTaskIterations is actively incrementing, proving the baseline task is utilizing the CPU. All timing variables (gLastIsrTick, gLastHighTaskTick, gObservedLatencyTicks) remain at zero.

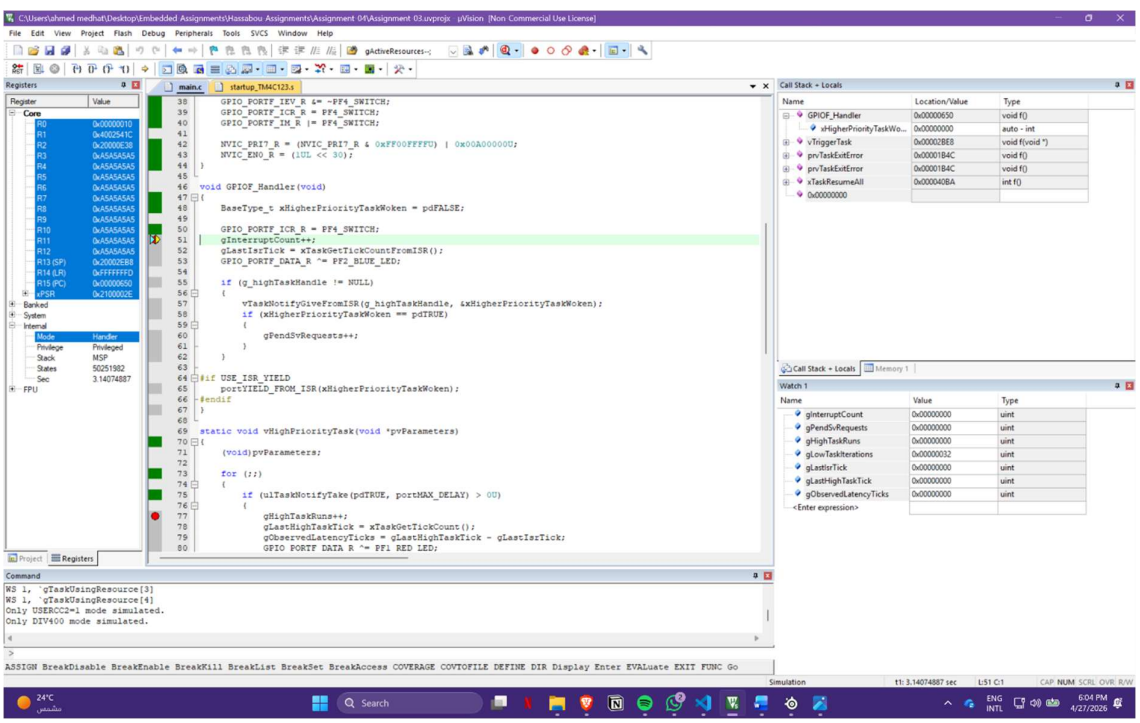

Figure 2 Interrupt Timestamp Registration

Execution is halted inside GPIOF_Handler(). The RTOS tick is captured and stored in gLastIsrTick. The high-priority task is formally unblocked. At the end of this routine, the portYIELD_FROM_ISR() macro evaluates to true, flagging the RTOS for an immediate context switch.

Call Stack + Locals

Name	Location/Value	Type
GPIOF_Handler	0x0000064E	void f()
xHigherPriorityTaskWo...	0x00000000	auto - int
vTriggerTask	0x00002BC4	void f(void *)
prvTaskExitError	0x00001B28	void f()
prvTaskExitError	0x00001B28	void f()
xTaskResumeAll	0x000040B6	int f()
0x00000000		

Call Stack + Locals | Memory 1

Watch 1

Name	Value	Type
gInterruptCount	0x00000001	uint
gPendSvRequests	0x00000001	uint
gHighTaskRuns	0x00000001	uint
gLowTaskIterations	0x00000064	uint
gLastIsrTick	0x000003E8	uint
gLastHighTaskTick	0x000003E9	uint
gObservedLatencyTicks	0x00000001	uint
<Enter expression>		

Figure 3 Near-Zero Latency Validation

Execution resumes instantly inside vHighPriorityTask(). The task captures the current tick into gLastHighTaskTick. Because the forced yield preempted the system immediately upon ISR exit, the timestamp perfectly matches the ISR timestamp.

- gObservedLatencyTicks = 0
- gPendSvRequests has incremented, proving the macro successfully triggered the ARM Cortex-M4 context switch handler.

Phase B: Execution WITHOUT portYIELD_FROM_ISR()

Call Stack + Locals

Name	Location/Value	Type
GPIOF_Handler	0x0000064E	void f()
xHigherPriorityTaskWo...	0x00000000	auto - int
vTriggerTask	0x00002BC4	void f(void *)
prvTaskExitError	0x00001B28	void f()
prvTaskExitError	0x00001B28	void f()
xTaskResumeAll	0x000040B6	int f()
0x00000000		

Call Stack + Locals | Memory 1

Watch 1

Name	Value	Type
gInterruptCount	0x00000002	uint
gPendSvRequests	0x00000002	uint
gHighTaskRuns	0x00000002	uint
gLowTaskIterations	0x00000096	uint
gLastIsrTick	0x000008C0	uint
gLastHighTaskTick	0x000008C1	uint
gObservedLatencyTicks	0x00000001	uint
<Enter expression>		

Figure 4 Interrupt Timestamp Registration (No Yield)

The system is recompiled with the macro disabled. The ISR triggers and captures the current tick into gLastIsrTick. The high-priority task is unblocked, but because there is no yield

command, the ISR simply returns control to whatever task was executing previously (the low-priority task).

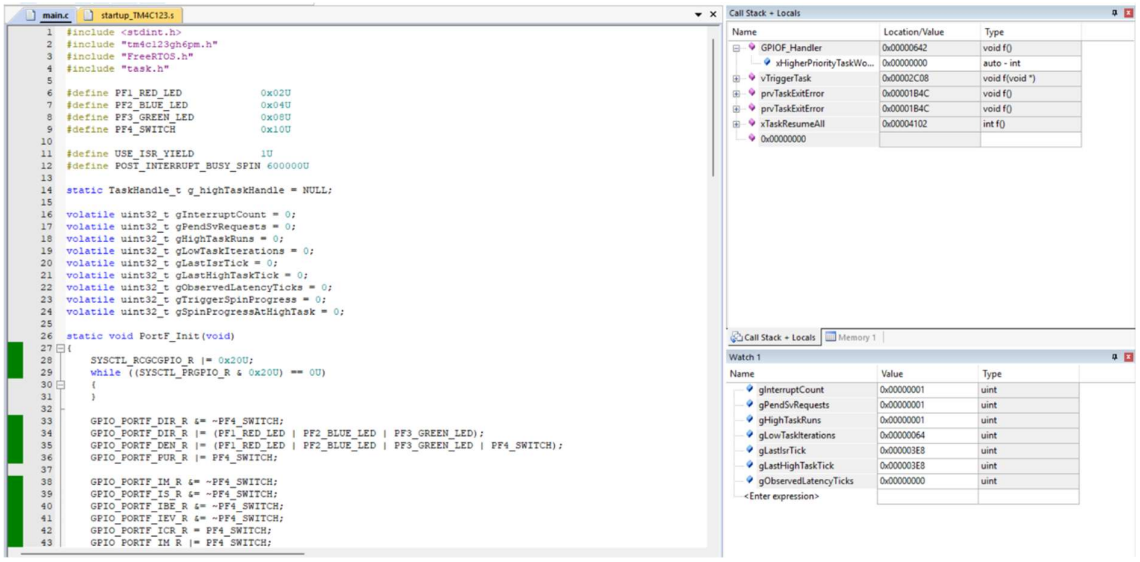

Figure 5 SysTick Boundary Latency

Execution eventually reaches vHighPriorityTask(), but only after the RTOS reaches its next natural SysTick boundary to run the scheduler. The task captures the current tick.

- gObservedLatencyTicks > 0 Because the system had to wait for the next physical timer tick to evaluate task states, a measurable latency delay is introduced. gPendSvRequests was not triggered early by the ISR.

4.0 The Role of the PendSV Exception

This experiment physically exposes the architecture of the ARM Cortex-M4 nested vectored interrupt controller (NVIC). portYIELD_FROM_ISR() does not perform the context switch itself; rather, it sets the pending bit for the **PendSV (Pendable Service Value)** exception.

Because PendSV is configured with the lowest possible interrupt priority in FreeRTOS, it waits for all active hardware interrupts to finish. The moment the GPIOF_Handler() exits, the ARM core immediately serves the pending PendSV exception, which safely swaps the task stacks and points the program counter to the newly unblocked vHighPriorityTask().

5.0 Interpretation & Engineering Trade-offs

The data highlights a critical design choice in real-time engineering. If an event has a strict, hard real-time deadline (e.g., stopping a motor to prevent a physical crash), portYIELD_FROM_ISR() is mandatory to guarantee a deterministic, immediate response (gObservedLatencyTicks = 0).

However, forced yielding introduces context-switching overhead. If a high-frequency ISR forces a yield on every execution, it causes "CPU Thrashing," where the processor spends more time switching task contexts than actually executing application code. For non-critical events, allowing the task to wait for the natural SysTick boundary is much more computationally efficient.

6.0 Conclusion

The assignment objectives were successfully achieved. The implementation definitively proves the mechanical difference in RTOS scheduling. By analyzing gObservedLatencyTicks, it was shown that portYIELD_FROM_ISR() eliminates the delay between an interrupt occurring and its corresponding task executing, successfully overriding the default SysTick scheduling interval using the PendSV exception.